JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class FileOutputStream

java.lang.Object
    java.io.OutputStream
        java.io.FileOutputStream

All Implemented Interfaces:

Closeable, Flushable, AutoCloseable

```
public class FileOutputStream
extends OutputStream
```

A file output stream is an output stream for writing data to a `File` or to a `FileDescriptor`. Whether or not a file is available or may be created depends upon the underlying platform. Some platforms, in particular, allow a file to be opened for writing by only one `FileOutputStream` (or other file-writing object) at a time. In such situations the constructors in this class will fail if the file involved is already open.

`FileOutputStream` is meant for writing streams of raw bytes such as image data. For writing streams of characters, consider using `FileWriter`.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the `try`-with-resources statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use `Cleaner` or some other mechanism.

Since:

1.0

See Also:

File,
FileDescriptor,
FileInputStream,
Files.newOutputStream(Path, OpenOption...)

Constructor Summary

Constructors

Constructor	Description
FileOutputStream(File file)	Creates a file output stream to write to the file represented by the specified `File` object.

FileOutputStream(FileDescriptor fdObj)	Creates a file output stream to write to the specified file descriptor, which represents an existing connection to an actual file in the file system.
FileOutputStream(File file, boolean append)	Creates a file output stream to write to the file represented by the specified `File` object.
FileOutputStream(String name)	Creates a file output stream to write to the file with the specified name.
FileOutputStream(String name, boolean append)	Creates a file output stream to write to the file with the specified name.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	**close**()	Closes this file output stream and releases any system resources associated with this stream.
FileChannel	**getChannel**()	Returns the unique `FileChannel` object associated with this file output stream.
final FileDescriptor	**getFD**()	Returns the file descriptor associated with this stream.
void	**write**(byte[] b)	Writes `b.length` bytes from the specified byte array to this file output stream.
void	**write**(byte[] b, int off, int len)	Writes `len` bytes from the specified byte array starting at offset `off` to this file output stream.

void	**write**(int b)	Writes the specified byte to this file output stream.

Methods declared in class OutputStream

flush, nullOutputStream

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

FileOutputStream

```
public FileOutputStream(String name)
                 throws FileNotFoundException
```

Creates a file output stream to write to the file with the specified name. If the file exists, it is truncated, otherwise a new file is created. Symbolic links are automatically redirected to the *target* of the link. A new `FileDescriptor` object is created to represent this file connection.

If the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason then a `FileNotFoundException` is thrown.

Implementation Requirements:

Invoking this constructor with the parameter `name` is equivalent to invoking `new FileOutputStream(name, false)`.

Parameters:

`name` - the system-dependent filename

Throws:

`FileNotFoundException` - if the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason

FileOutputStream

```
public FileOutputStream(String name,
                        boolean append)
                 throws FileNotFoundException
```

Creates a file output stream to write to the file with the specified name. If the file exists, it is truncated unless the second argument is `true`, in which case bytes will be written to the end of the file rather than the beginning. If the file does not exist, it is created. Symbolic links are automatically redirected to the *target* of the link. A new `FileDescriptor` object is created to represent this file connection.

If the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason then a `FileNotFoundException` is thrown.

Parameters:

`name` - the system-dependent file name

`append` - if `true`, then bytes will be written to the end of the file rather than the beginning

Throws:

`FileNotFoundException` - if the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason.

Since:

1.1

FileOutputStream

```
public FileOutputStream(File file)
                 throws FileNotFoundException
```

Creates a file output stream to write to the file represented by the specified `File` object. If the file exists, it is truncated, otherwise a new file is created. Symbolic links are automatically redirected to the *target* of the link. A new `FileDescriptor` object is created to represent this file connection.

If the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason then a `FileNotFoundException` is thrown.

Parameters:

`file` - the file to be opened for writing.

Throws:

`FileNotFoundException` - if the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason

See Also:

`File.getPath()`

FileOutputStream

```
public FileOutputStream(File file,
                        boolean append)
                 throws FileNotFoundException
```

Creates a file output stream to write to the file represented by the specified `File` object. If the file exists, it is truncated unless the second argument is `true`, in which case bytes will be written to the end of the file rather than the beginning. If the file does not exist, it is created. Symbolic links are automatically redirected to the *target* of the link. A new `FileDescriptor` object is created to represent this file connection.

If the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason then a `FileNotFoundException` is thrown.

Parameters:

`file` - the file to be opened for writing.

`append` - if `true`, then bytes will be written to the end of the file rather than the beginning

Throws:

`FileNotFoundException` - if the file exists but is a directory rather than a regular file, does not exist but cannot be created, or cannot be opened for any other reason

Since:

1.4

See Also:

`File.getPath()`

FileOutputStream

```
public FileOutputStream(FileDescriptor fdObj)
```

Creates a file output stream to write to the specified file descriptor, which represents an existing connection to an actual file in the file system.

If `fdObj` is null then a `NullPointerException` is thrown.

This constructor does not throw an exception if `fdObj` is `invalid`. However, if the methods are invoked on the resulting stream to attempt I/O on the stream, an `IOException` is thrown.

Parameters:

`fdObj` - the file descriptor to be opened for writing

Method Details

write

```
public void write(int b)
           throws IOException
```

Writes the specified byte to this file output stream. Implements the `write` method of `OutputStream`.

Specified by:

`write` in class `OutputStream`

Parameters:

`b` - the byte to be written.

Throws:

`IOException` - if an I/O error occurs.

write

```
public void write(byte[] b)
           throws IOException
```

Writes `b.length` bytes from the specified byte array to this file output stream.

Overrides:

`write` in class `OutputStream`

Parameters:

`b` - the data.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`OutputStream.write(byte[], int, int)`

write

```
public void write(byte[] b,
                  int off,
                  int len)
           throws IOException
```

Writes `len` bytes from the specified byte array starting at offset `off` to this file output stream.

Overrides:

`write` in class `OutputStream`

Parameters:

`b` - the data.

`off` - the start offset in the data.

`len` - the number of bytes to write.

Throws:

`IOException` - if an I/O error occurs.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

close

```
public void close()
           throws IOException
```

Closes this file output stream and releases any system resources associated with this stream. This file output stream may no longer be used for writing bytes.

If this stream has an associated channel then the channel is closed as well.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `OutputStream`

API Note:

Overriding `close()` to perform cleanup actions is reliable only when called directly or when called by try-with-resources.

Implementation Requirements:

Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use the `Cleaner` mechanism.

If this stream has an associated channel then this method will close the channel, which in turn will close this stream. Subclasses that override this method should be prepared to handle possible reentrant invocation.

Throws:

`IOException` - if an I/O error occurs.

getFD

```
public final FileDescriptor getFD()
                           throws IOException
```

Returns the file descriptor associated with this stream.

Returns:

the `FileDescriptor` object that represents the connection to the file in the file system being used by this `FileOutputStream` object.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`FileDescriptor`

getChannel

```
public FileChannel getChannel()
```

Returns the unique `FileChannel` object associated with this file output stream.

The initial `position` of the returned channel will be equal to the number of bytes written to the file so far unless this stream is in append mode, in which case it will be equal to the size of the file. Writing bytes to this stream will increment the channel's position accordingly. Changing the channel's position, either explicitly or by writing, will change this stream's file position.

Returns:

the file channel associated with this file output stream

Since:

1.4

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.